Deployment Strategy

API-First Azure Hosting, CI/CD, Security & Operations

API-FIRST ARCHITECTURE | ASP.NET CORE | MICROSOFT AZURE

Status: Proposed design package — source code not supplied for implementation verification

Document Control

Project	MediCare — Clinic Management & Appointment System
Phase	Phase 5 — Deployment & Integration
Architecture	API-First N-Tier Web Application
Version	1.0 — Proposed baseline

How to Read This Document

Items marked CONFIRMED are explicit project decisions or requirements. Items marked PROPOSED are recommended technical choices that require approval or implementation. Items marked OPTIONAL depend on scope and schema approval. Items marked OPEN require a team decision before the related implementation is treated as final.

1. Executive Summary

MediCare is deployed as two independently deployable applications: MediCare.Web (ASP.NET Core MVC presentation client) and MediCare.API (ASP.NET Core Web API backend). The API owns all authoritative business logic, Identity, authorization, database access, SignalR hubs, background jobs and provider integrations. MediCare.Web consumes the API over HTTPS using a server-side Typed HttpClient. The browser never receives a database connection string and the MVC application never connects directly to the business database.

Deployment baseline — Use separate Azure App Service resources for Web and API in each environment. For the graduation-project budget, the two apps may share one App Service Plan within an environment, but Staging and Production must be isolated from one another.

2. Target Topology

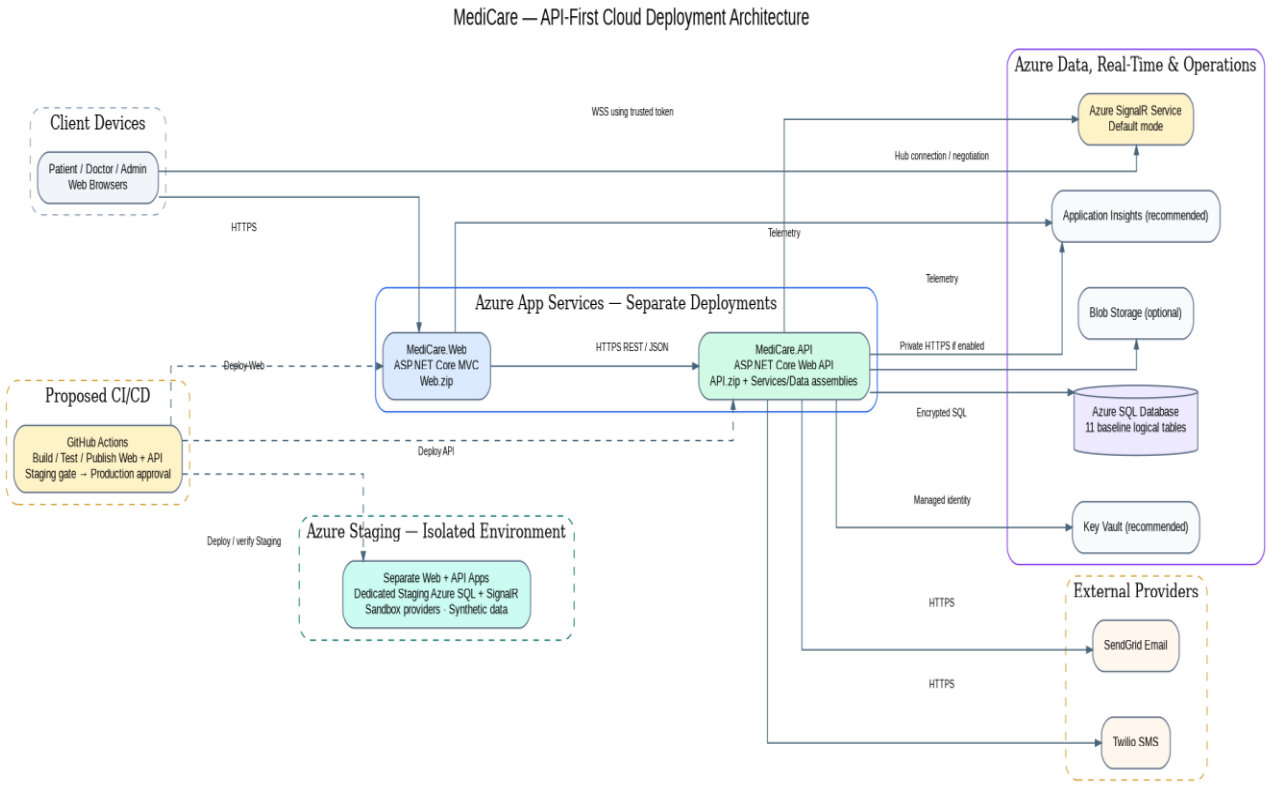


Figure 1. Logical cloud deployment topology. The editable native diagram is supplied separately as MediCare_Deployment_Diagram.drawio.

3. Environment Model

Environment	Application resources	Database / providers	Access
Development	Local MediCare.Web + MediCare.API	Local SQL Server Developer/approved dev DB; provider mocks or sandbox	Developers only; synthetic data
Staging	Separate Web App Service + API App Service	Dedicated Staging Azure SQL DB; Staging SignalR; Twilio/SendGrid test credentials	Team + testers; synthetic/anonymized data
Production	Separate Web App Service + API App Service	Production Azure SQL DB; Production SignalR; production provider credentials	Public users; least privilege

Keep environment-specific API base URLs, database connections, token-signing keys, provider credentials, and SignalR connection strings separate. Never copy Production data into development or ordinary staging environments.

4. Azure Resource Selection

Resource	Role	Status / decision
Azure App Service — MediCare.Web	Hosts MVC presentation app	Required in target cloud topology
Azure App Service — MediCare.API	Hosts backend API	Required in target cloud topology
Azure App Service Plan	Compute plan; can host both app resources in same environment	Proposed cost-conscious option; size after load test
Azure SQL Database	Managed relational database	Documented cloud database target
Azure SignalR Service	Scales real-time connections; API remains hub owner	Documented; use Default mode for ASP.NET Core server-hosted hubs
Twilio	SMS provider	Documented external provider; use staging/test credentials
SendGrid	Email provider	Documented external provider; use staging/test credentials
Azure Key Vault	Secret management	Recommended; prefer managed identity from App Service
Application Insights / Azure Monitor	Application telemetry and diagnostics	Recommended
Azure Blob Storage	Private storage for uploaded medical documents	Optional only if file uploads are approved
GitHub Actions	Build/test/deploy pipeline	Proposed primary CI/CD choice; confirm repository host

5. Deployment Diagram Notes

- Browser → MediCare.Web: HTTPS.
- MediCare.Web server → MediCare.API: HTTPS REST/JSON with server-side credentials.
- MediCare.API → Azure SQL: encrypted SQL connection; database firewall/private networking policy must be selected.
- MediCare.API → Azure SignalR: server-side connection in Default mode; browser connects using a trusted negotiation result and short-lived hub access token.
- MediCare.API → Twilio/SendGrid: outbound HTTPS through backend provider adapters.
- MediCare.API → Key Vault: managed identity where supported; no hard-coded secrets.
- MediCare.Web has no database connection and no direct provider credentials.

6. Component Deployment and Release Order

1. Provision resource groups, App Service Plan(s), two App Service resources, Azure SQL, SignalR and optional supporting resources.
2. Create separate Development, Staging and Production configuration sets. Configure API base URL in MVC as environment-specific application configuration.
3. Build the solution and run unit tests for MediCare.Services. Run backend integration tests against a disposable or isolated SQL Server database.
4. Generate separate artifacts for MediCare.API and MediCare.Web. Ensure Services and Data class libraries are packaged as dependencies of the API, not separately deployed cloud services.
5. Apply reviewed database migrations through the backend deployment process only. Prefer a controlled migration step or reviewed idempotent script; do not let MVC run migrations.
6. Deploy MediCare.API to Staging. Verify health, database connectivity, Identity, SignalR negotiation, background processing and provider sandbox configuration.
7. Deploy MediCare.Web to Staging with the Staging API base URL. Verify login/session flow and all MVC-to-API calls.
8. Run API contract tests, integration tests, end-to-end smoke tests, security checks and UAT. Approve release only after critical tests pass.
9. Deploy the same immutable API and MVC artifacts to Production after approval. Apply only reviewed production migrations and verify health endpoints.
10. Run production smoke tests, check logs/metrics and confirm notifications. Keep the previous application artifact available for rollback.

7. CI/CD — Proposed GitHub Actions

GitHub Actions is selected as the proposed primary CI/CD system, subject to confirmation that the project repository is hosted on GitHub. The pipeline must build and test the solution, then publish two independent artifacts and deploy them separately. If the repository is hosted in Azure Repos, use Azure Pipelines instead without changing the target architecture.

Stage	Actions	Gate / output
Pull request validation	Restore; build; format/analyzer check; unit tests; API contract lint; secret scan	Block merge on failure
Build artifacts	Publish MediCare.API and MediCare.Web separately; attach version/commit metadata	Immutable Web.zip + API.zip
Staging deployment	Deploy API; apply approved migration; health check; deploy MVC; smoke tests	Staging acceptance gate
Production approval	Manual approval; verify artifact digest and release notes	Authorized reviewer
Production deployment	Deploy API and Web separately; migration approval; health checks	Release record
Post-deployment	Smoke tests; synthetic login/search/booking checks; telemetry review	Rollback decision if critical failures

8. Configuration and Secrets

- Use ASPNETCORE_ENVIRONMENT values Development, Staging and Production as appropriate.
- Store API base URL, feature flags, timeout settings and non-secret environment settings in app configuration.
- Store SQL connection strings, JWT signing material, SignalR connection strings, Twilio credentials and SendGrid API key in Key Vault or protected App Service settings; Key Vault + managed identity is recommended.
- Never commit secrets to Git, appsettings.json, Postman collection, OpenAPI examples or logs.
- Restrict CORS to exact approved origins only if browser code calls the API directly. If MVC server-to-server calls the API, CORS is not needed for that HTTP path; the SignalR browser path must be evaluated separately.
- Apply least-privilege identities and rotate credentials when staff or environments change.

9. Authentication and SignalR Deployment

The API owns ASP.NET Core Identity and authorization. The MVC site maintains its own secure browser session and stores API credentials server-side. Proposed API-to-MVC authentication uses short-lived JWT access tokens and a protected refresh mechanism, but this must be confirmed against source code and reviewed before implementation. The browser must not receive long-lived refresh credentials.

For Azure SignalR, use Default service mode because the Hub logic remains hosted in MediCare.API and Azure SignalR acts as the connection proxy. Do not use Serverless mode for an ASP.NET Core-hosted hub. Clients obtain connection information from a trusted server negotiation step; clients must not construct their own service access tokens. (Official Microsoft documentation: https://learn.microsoft.com/en-us/azure/azure-signalr/signalr-quickstart-dotnet-core and https://learn.microsoft.com/en-us/azure/azure-signalr/signalr-concept-client-negotiation)

10. Database Migration Strategy

- Create migrations from MediCare.Data using the selected EF Core version. Review generated SQL and constraints, especially appointment interval overlap, unique MedicalRecords.AppointmentId and unique Prescriptions(MedicalRecordId, VersionNumber).
- Apply schema changes to Staging first and run regression/integration tests.
- For production, create/verify a restorable backup point, approve migration, then apply through a dedicated backend release step.
- Prefer expand-and-contract changes for changes that span API/Web release versions: add nullable/new fields first, deploy compatible code, backfill, then enforce constraints in a later release.
- Do not assume Down migrations are safe for data-destructive changes. Define forward-fix or restore procedure for each risky migration.

- Only MediCare.API/backend deployment owns migrations; MediCare.Web must not reference or invoke DbContext/migration commands.

11. Appointment Concurrency and Transaction Integrity

The project requires preventing overlapping appointments even when requests arrive simultaneously. RowVersion is useful for detecting concurrent updates to an existing row, but by itself it does not prevent two transactions from inserting different appointment rows whose time intervals overlap. Implement a transactional interval check with an appropriate SQL Server concurrency strategy (for example, a transaction with SERIALIZABLE isolation and suitable indexed range locking, or an approved slot-reservation model with a unique constraint). Validate the full interval condition: existing.StartDateTime < requested.EndDateTime AND existing.EndDateTime > requested.StartDateTime. Test simultaneous requests against the real SQL Server engine, not EF InMemory.

12. Background Processing

A backend background worker must process: (a) Pending appointments that have exceeded the 24-hour response window, and (b) reminders scheduled 24 hours before an appointment. If the worker runs inside App Service, use an Always On-capable plan and durable idempotent processing. When multiple API instances are used, prevent duplicate execution using a database lease/claim or equivalent distributed coordination. Do not rely on an in-memory timer alone for durable scheduling.

13. Notification Reliability

Persist an in-app notification record and publish the SignalR event as a best-effort real-time signal. SMS/email failures must be retried with bounded exponential backoff and recorded delivery state. The current Notifications baseline does not contain explicit provider delivery status, retry count, next-attempt time or channel preference fields; either extend Notifications or approve an outbox/delivery table and preferences storage before promising reliable retries/preferences as implemented.

14. Medical Documents

File uploads are a Should-priority requirement. If included, store file bytes in private Azure Blob Storage, not as public files. Store metadata (owner, medical record, blob identifier, content type, size, checksum, upload timestamp) in an approved MedicalDocuments schema extension. Validate content type and file signature, enforce file size limits, authorize every download, use short-lived access URLs or an authorized API streaming endpoint, and log access. Do not enable Blob Storage in Production until this design and retention rules are approved.

15. Monitoring, Audit and Privacy

- Use Application Insights/Azure Monitor for application traces, availability, dependency failures and latency. Correlate requests with traceId across MVC and API.
- Keep audit events for each medical record read/change in a dedicated protected audit store/table or approved immutable logging facility. Application telemetry alone is not a substitute for a business audit trail.
- Never log passwords, access/refresh tokens, provider keys, full medical record contents or unnecessary PHI. Mask identifiers where practical.
- Alert on API 5xx spikes, SQL dependency failures, background worker lag, provider retry exhaustion, SignalR connection errors and failed health checks.
- Privacy/legal compliance must be assessed against applicable Egyptian and other relevant data-protection/health-data obligations; this architecture does not itself certify legal compliance.

16. Backup, Recovery and Availability

Azure SQL Database provides service-managed automated backups and point-in-time restore. The exact schedule is managed by Azure; customers configure retention and redundancy rather than scheduling a daily full backup job themselves. Microsoft documentation states that PITR retention is configurable and that long-term retention can be configured separately. Confirm the selected tier's defaults and configure retention to meet the project's data-protection policy. Conduct restore drills and record the measured restore time.

Control	Initial proposal	Approval required
Short-term PITR retention	35 days where tier supports it; validate cost and service limits	Yes
Long-term retention	Not enabled until academic/legal retention needs are decided	Yes
RPO	Proposed target ≤ 1 hour for application data, subject to tier/recovery design	Yes
RTO	Proposed target ≤ 8 hours for initial graduation deployment; validate by restore drill	Yes
Availability	99.5% is a project target, not a guarantee; App Service/SQL tiers and operational design determine achievable SLO	Yes

17. Rollback and Incident Response

- Application rollback: redeploy previous immutable API and/or MVC artifact independently.
- Configuration rollback: restore previous versioned App Service settings/Key Vault references; rotate any exposed credentials.
- Database rollback: prefer forward-fix for additive changes; restore to a new database using PITR when data corruption/destructive migration requires recovery, then switch connection configuration after verification.
- Provider outage: preserve notification work, retry within bounded policy, surface degraded status, and avoid blocking booking transaction completion on email/SMS delivery.
- After rollback: run health checks, authentication, doctor search, booking conflict smoke test, medical-record access authorization and notification checks.

18. Production Release Checklist

Check	Owner	Evidence / status
API and Web artifacts built from same approved release commit	DevOps	Commit SHA + artifact digest
Staging tests passed; critical defects closed	QA	Test run report
Migration reviewed and backup/restore point verified	DBA	Migration review + restore point
API health, SQL and SignalR verified	Backend / DevOps	Health checks
Twilio/SendGrid production credentials configured	DevOps	Secret reference only; no key printed
MVC API base URL points to Production API	Frontend / DevOps	Environment configuration
CORS/origin and HTTPS settings reviewed	Security	Configuration review
Admin seed/bootstrap secure and public Admin signup disabled	Backend	Provisioning evidence
Smoke tests complete	QA	Release test report
Rollback artifacts and runbook available	DevOps	Rollback rehearsal

19. Open Decisions Before Production

ID	Decision	Why it matters
DEC-01	Identity physical mapping: Users vs AspNetUsers	Avoid duplicate account stores and broken profile FKs.
DEC-02	Doctor location field	Doctor discovery requirement includes location but baseline schema lacks a clear location field.

ID	Decision	Why it matters
DEC-03	Slot length source	DoctorSchedules physical model does not clearly show slot length; ClinicSettings is Should/optional.
DEC-04	Appointment reason/notes/type columns	Requirements include reason/notes and type as Should; schema support must be confirmed.
DEC-05	Appointment interval concurrency implementation	RowVersion alone does not prevent overlapping inserts.
DEC-06	Audit log persistence	NFR requires audit on record open/change; baseline schema lacks AuditLogs.
DEC-07	Notification outbox/status/preferences	Baseline Notifications lacks retry/channel delivery state.
DEC-08	No-show policy and reschedule status behavior	State/transition rules require final business decision.
DEC-09	RPO/RTO and legal retention	Backup policy and recovery targets must be approved.
DEC-10	Actual repository host and CI/CD provider	GitHub Actions is proposed pending repository confirmation.

20. Official Technical References

Reference	Official URL
Microsoft .NET support policy	https://dotnet.microsoft.com/en-us/platform/support/policy/dotnet-core
EF Core 10 release and requirements	https://learn.microsoft.com/en-us/ef/core/what-is-new/ef-core-10.0/whatsnew
EF Core release support matrix	https://learn.microsoft.com/en-us/ef/core/what-is-new/index
ASP.NET Core 10 documentation	https://learn.microsoft.com/en-us/aspnet/core/?view=aspnetcore-10.0
ASP.NET Core Web API guidance	https://learn.microsoft.com/en-us/aspnet/core/web-api
Azure SignalR Quickstart / Default mode	https://learn.microsoft.com/en-us/azure/azure-signalr/signalr-quickstart-dotnet-core
Azure SignalR client negotiation	https://learn.microsoft.com/en-us/azure/azure-signalr/signalr-concept-client-negotiation
SQL Server 2022 lifecycle	https://learn.microsoft.com/en-us/lifecycle/products/sql-server-2022
Azure SQL automated backups	https://learn.microsoft.com/en-us/azure/azure-sql/database/automated-backups-overview?view=azuresql
Azure SQL recovery / PITR	https://learn.microsoft.com/en-us/azure/azure-sql/database/recovery-using-backups?view=azuresql